

FarmConnect Project Report

Auto-generated from docs/FarmConnect_Project_Report.md

FarmConnect Project Report

Date: 2026-03-27

1. Project Summary

FarmConnect is a full-stack AgriTech marketplace platform that connects agricultural producers directly with buyers. The system supports a bilingual UI (English/Arabic), role-based access control, product discovery and management, checkout and payment flows, delivery tracking, notifications, and admin verification workflows.

The project is implemented as a monorepo with:

- Web frontend (`apps/web`) built with React + TypeScript + Vite + Tailwind.
- API backend (`apps/api`) built with Express + TypeScript + Prisma + PostgreSQL.
- AI microservice (`apps/ai-service`) built with Python + FastAPI (currently health/service scaffold).
- Shared workspace packages for types, schemas, constants, configs, and UI primitives.

2. Technology Stack

- Frontend: React 19, React Router 7, TanStack Query, i18next, Tailwind CSS, lucide-react.
- Backend: Express 5, Prisma ORM, PostgreSQL, Zod, JWT, bcrypt, Helmet, CORS, rate limiting.
- AI Service: FastAPI (Python), Pydantic settings pattern.
- Monorepo Tooling: pnpm workspaces, Turborepo.
- Testing: Vitest (API tests active, web test harness configured).
- Build/Runtime: TypeScript, Vite, tsx watch.

3. Exact Workspace Structure

3.1 Root

```
```text
farmconnect/
 .editorconfig
 .env.example
 .gitattributes
 .gitignore
 .npmrc
 .nvmrc
 .github/
 .turbo/
 apps/
 docs/
 infra/
 scripts/
 packages/
 LICENSE
 README.md
 package.json
 pnpm-lock.yaml
 pnpm-workspace.yaml
 tsconfig.base.json
 turbo.json
```

```
3.2 apps/
``text
apps/
 api/
 web/
 ai-service/
``
```

```
3.3 apps/api
``text
apps/api/
 .env
 .env.example
 Dockerfile
 package.json
 tsconfig.json
 vitest.config.ts
 prisma/
 migrations/
 schema.prisma
 seed.ts
 tests/
 health.test.ts
 setup.ts
 src/
 app.ts
 index.ts
 server.ts
 config/
 index.ts
 routes.ts
 core/
 errors/
 AppError.ts
 HttpError.ts
 index.ts
 logger/
 index.ts
 pagination/
 index.ts
 response/
 index.ts
 middleware/
 authenticate.ts
 authorize.ts
 errorHandler.ts
 httpLogger.ts
 index.ts
 notFound.ts
 rateLimiter.ts
 requestId.ts
 modules/
 auth/
 auth.controller.ts
```

- auth.repository.ts
- auth.router.ts
- auth.schemas.ts
- auth.service.ts
- cart/
  - cart.controller.ts
  - cart.repository.ts
  - cart.router.ts
  - cart.schemas.ts
  - cart.service.ts
- delivery/
  - delivery.controller.ts
  - delivery.repository.ts
  - delivery.router.ts
  - delivery.schemas.ts
  - delivery.service.ts
- health/
  - health.router.ts
- notifications/
  - notifications.channels.ts
  - notifications.controller.ts
  - notifications.repository.ts
  - notifications.router.ts
  - notifications.schemas.ts
  - notifications.service.ts
- orders/
  - orders.controller.ts
  - orders.repository.ts
  - orders.router.ts
  - orders.schemas.ts
  - orders.service.ts
- payments/
  - payments.controller.ts
  - payments.repository.ts
  - payments.router.ts
  - payments.schemas.ts
  - payments.service.ts
  - payments.webhook.router.ts
- providers/
  - baridimob.provider.ts
  - payment-provider.ts
  - stripe.provider.ts
- product-images/
  - image-storage.service.ts
  - product-image.controller.ts
  - product-image.repository.ts
  - product-image.router.ts
  - product-image.schemas.ts
  - product-image.service.ts
- products/
  - product.controller.ts
  - product.repository.ts
  - product.router.ts
  - product.schemas.ts
  - product.service.ts
- profile/

```
 profile.controller.ts
 profile.repository.ts
 profile.router.ts
 profile.schemas.ts
 profile.service.ts
 prisma/
 client.ts
 types/
 express.d.ts
 ...
```

### ### 3.4 apps/web

```
``text
apps/web/
 .env.example
 index.html
 package.json
 postcss.config.js
 tailwind.config.ts
 tsconfig.json
 tsconfig.node.json
 vite.config.ts
 vitest.config.ts
 public/
 locales/
 en/common.json
 ar/common.json
 logo.svg
 robots.txt
 tests/
 setup.ts
 src/
 main.tsx
 vite-env.d.ts
 app/
 App.tsx
 layouts/
 AdminLayout.tsx
 AuthLayout.tsx
 DashboardLayout.tsx
 MainLayout.tsx
 providers/
 AuthProvider.tsx
 I18nProvider.tsx
 NotificationProvider.tsx
 QueryProvider.tsx
 ThemeProvider.tsx
 router/
 index.tsx
 route-guards.tsx
 routes/
 admin.routes.tsx
 buyer.routes.tsx
 dashboard.routes.tsx
 producer.routes.tsx
 public.routes.tsx
```

```
store/
 index.ts
 slices/auth.slice.ts
features/
 admin/pages/AdminDashboardPage.tsx
 analytics/pages/BuyerDashboardPage.tsx
 analytics/pages/ProducerDashboardPage.tsx
 auth/
 api/auth.api.ts
 pages/LoginPage.tsx
 pages/RegisterPage.tsx
 cart/
 api/cart.api.ts
 hooks/useCart.ts
 pages/CartPage.tsx
 notifications/
 api/notifications.api.ts
 components/NotificationPanel.tsx
 hooks/useNotifications.ts
 pages/NotificationsPage.tsx
 orders/
 api/orders.api.ts
 components/DeliveryStatusTimeline.tsx
 components/MapTrackerPlaceholder.tsx
 components/OrderStatusBadge.tsx
 hooks/useOrders.ts
 pages/BuyerOrderDetailPage.tsx
 pages/BuyerOrdersPage.tsx
 pages/CheckoutPage.tsx
 pages/OrderTrackingPage.tsx
 pages/ProducerOrderDetailPage.tsx
 pages/ProducerOrdersPage.tsx
 utils/order.utils.ts
 products/
 api/products.api.ts
 components/ProductCard.tsx
 components/ProductForm.tsx
 hooks/useProducts.ts
 pages/AddProductPage.tsx
 pages/EditProductPage.tsx
 pages/MyProductsPage.tsx
 pages/ProductDetailsPage.tsx
 pages/ProductListPage.tsx
 utils/product.utils.ts
 profile/
 api/profile.api.ts
 hooks/useProfile.ts
 pages/AdminVerificationReviewPage.tsx
 pages/BuyerProfilePage.tsx
 pages/ProducerOnboardingPage.tsx
 pages/ProducerProfilePage.tsx
 pages/ProducerVerificationPage.tsx
 index.ts
shared/
 api/client.ts
 components/LanguageToggle.tsx
```

```
 components/ThemeToggle.tsx
 pages/AppErrorPage.tsx
 pages/NotFoundPage.tsx
 utils/api-error.ts
 styles/index.css
 ...
```

### ### 3.5 apps/ai-service

```
  ```text
  apps/ai-service/
    .env.example
    Dockerfile
    main.py
    README.md
    requirements.txt
    src/
      app.py
      config.py
      api/health.py
      contracts/__init__.py
      utils/logger.py
    tests/
  ...
```

3.6 packages/

```
  ```text
 packages/
 shared/
 package.json
 tsconfig.json
 tsconfig.esm.json
 src/
 index.ts
 constants/index.ts
 enums/index.ts
 schemas/index.ts
 types/index.ts
 ui/
 package.json
 tsconfig.json
 src/
 index.ts
 components/Button.tsx
 components/Spinner.tsx
 config/
 package.json
 eslint/
 base.js
 node.js
 react.js
 prettier/
 index.js
 typescript/
 base.json
 node.json
 react.json
```

...

### ### 3.7 Other Important Folders

- `docs/`: architecture and documentation.
- `infra/`: infrastructure-level notes/assets (currently minimal in repo).
- `scripts/`: operational scripts (`reset-db.sh`, `seed.ts`).

## ## 4. Folder/File Purpose Breakdown

### ### 4.1 Backend API (`apps/api/src`)

- `app.ts`: express app wiring, middleware chain, CORS, routes, error handling.
- `index.ts` / `server.ts`: server bootstrap.
- `config/routes.ts`: central API route registration under `/api`.
- `core/errors/\*`: consistent typed error framework.
- `middleware/\*`: auth, role checks, logging, request ids, rate limiting, error/not-found handling.
- `modules/\*`: vertical business modules using controller-service-repository layering.
- `modules/payments/providers/\*`: gateway abstraction and provider implementations.
- `modules/notifications/notifications.channels.ts`: channel policy routing for notifications.
- `prisma/client.ts`: singleton Prisma client.

### ### 4.2 Frontend (`apps/web/src`)

- `app/layouts/\*`: main shell layouts for public/auth/dashboard/admin.
- `app/providers/\*`: auth session state, query client, theme, i18n, notifications context.
- `app/router/\*`: role-aware route maps and guards.
- `features/\*`: domain-first modules (auth, products, orders, profile, etc.).
- `shared/\*`: reusable app-level utility and base components.
- `i18n/index.ts`: i18next initialization and language detection/persistence.

### ### 4.3 Shared Packages

- `packages/shared`: source of truth for shared constants, enums, API/pagination types, and base schemas.
- `packages/ui`: reusable button/spinner primitives for consistent UX foundations.
- `packages/config`: centralized eslint/prettier/typescript presets for workspace consistency.

## ## 5. Implemented System Functionalities

### ### 5.1 Authentication and Authorization

- Registration and login flows.
- Role-based access (`BUYER`, `PRODUCER`, `ADMIN`).
- Guarded dashboard routes by role.
- Access/refresh token architecture on API side.

### ### 5.2 Profiles and Producer Lifecycle

- Buyer and producer profile management.
- Producer onboarding and verification pages.
- Admin verification review workflow.
- Profile avatar URL support and random avatar helper in profile pages.

### ### 5.3 Product Management and Catalog

- Public product listing with filtering, sorting, pagination.
- Product details with image gallery, thumbnail selection, swipe navigation, and image counter.
- Producer product CRUD (`MyProducts`, create, edit, archive/soft delete path).
- Product images module on API side.
- Seeded realistic produce images for demo data.

### ### 5.4 Cart and Ordering

- Buyer cart add/update/remove flows.
- Checkout from cart to order creation.
- Buyer orders list and detail pages.
- Producer orders list/detail pages with status transitions.
- Status-aware badges for order/payment/delivery lifecycle.

### ### 5.5 Payments

- Payment intent creation flow per order.
- Provider abstraction (`STRIPE`, `BARIDIMOB`).
- Stripe webhook route support.
- Fallback behavior: if Stripe is unavailable, flow can continue using BaridiMob.
- Order/payment status checks and payment event timeline display.

### ### 5.6 Delivery Tracking

- Delivery updates with status, location, and notes.
- Timeline visualization and map placeholder component.
- Buyer and producer visibility into tracking progression.

### ### 5.7 Notifications

- In-app notifications API and frontend panel/pages.
- Unread counters and read-state behavior.
- Notifications emitted for key events: order updates, payment outcomes, delivery updates.

### ### 5.8 Localization and Theming

- Arabic/English switching using i18next.
- Arabic locale support expanded across product specs, order pages, auth feedback, and tracking text.
- RTL-compatible language handling through theme/i18n providers.
- Dark mode support across public/dashboard/auth/admin layouts and major feature pages.

### ### 5.9 UX/Resilience Enhancements Added

- Product details now includes producer profile info/photo card button.
- Registration has real-time password requirement checklist and per-field validity indicators (check/X).
- Removed hardcoded demo credentials from login page.
- Added custom app error page usage in route error elements.

## ## 6. API Module Responsibilities

- `auth`: login, register, token/session flows.
- `profile`: buyer/producer profile read/update and related profile operations.
- `products`: public catalog and producer-managed product operations.
- `product-images`: product media attachment management.
- `cart`: buyer shopping cart operations.
- `orders`: checkout, order listing/detail, producer transitions.
- `payments`: payment intents, status, providers, webhook handling.
- `delivery`: tracking retrieval and producer update creation.
- `notifications`: list/read and unread handling.
- `health`: liveness endpoint.

## ## 7. Database Schema (Prisma/PostgreSQL)

### ### 7.1 Core Enums

- Roles/identity: `Role`, `UserStatus`, `VerificationStatus`.
- Commerce: `ProductStatus`, `OrderStatus`, `PaymentStatus`, `DeliveryStatus`.
- Payments/finance: `PaymentMethod`, `PaymentEventType`, `PayoutStatus`.
- Notifications/audit: `NotificationType`, `NotificationChannel`, `AuditAction`.



### ### 7.2 Main Models

- `User`: central identity, role, status, avatar, relations.
- `RefreshToken`: refresh token storage and revocation metadata.
- `ProducerProfile`, `BuyerProfile`: role-specific profile details.
- `ProducerVerificationRequest`: document-based producer verification flow.
- `Address`: buyer saved addresses and order address relation.
- `Category`: hierarchical product categories with localized names.
- `Product`: producer-owned product record with localization fields.
- `ProductImage`: ordered image list per product.
- `Cart`, `CartItem`: buyer cart and line items.
- `Order`, `OrderItem`: immutable purchase records with product snapshots.
- `Payment`, `PaymentEvent`: payment state and gateway event audit trail.
- `Payout`: producer payout tracking.
- `DeliveryTracking`: delivery timeline updates.
- `Notification`: in-app/email/SMS/push event records.
- `AnalyticsDaily`: daily metrics aggregation.
- `AiRecommendation`, `AiForecast`: AI recommendation/forecast entities.
- `AuditLog`: actor-target action history with JSON diffs.

### ### 7.3 Key Relationship Notes

- One user can have one buyer profile or one producer profile.
- One buyer has one cart; cart has many cart items.
- Orders belong to buyers and contain immutable item snapshots.
- Each order has at most one payment record; payment has many events.
- Products belong to producer profiles; product images are one-to-many.
- Notifications are per user and indexed by read/time dimensions.

## ## 8. Frontend Route Map (Implemented)

### ### Public

- `/`
- `/products`
- `/products/:slug`
- `/login`
- `/register`

### ### Dashboard (Buyer/Producer role-gated)

- Shared: `/dashboard`, `/dashboard/profile`, `/dashboard/orders`, `/dashboard/orders/:orderId`, `/dashboard/orders/:orderId/tracking`, `/dashboard/notifications`
- Buyer-only: `/dashboard/cart`, `/dashboard/checkout`
- Producer-only: `/dashboard/onboarding`, `/dashboard/verification`, `/dashboard/products`, `/dashboard/products/new`, `/dashboard/products/:productId/edit`

### ### Admin

- `/admin`
- `/admin/producer-verifications`

## ## 9. Quality, Testing, and Runtime Status

- API test suite passing (`health.test.ts`).
- Web typecheck passing.
- API typecheck passing.
- Web test command configured and currently reports no test files.
- Dev stack runs via `pnpm dev` (API at 4000, web Vite port as available).

## ## 10. Current Known Constraints

- Stripe full success path depends on complete gateway keys and webhook configuration in environment.

- AI service is scaffolded with health/config/logger foundations; advanced AI modules are planned in later phases.

#### ## 11. Recommended Next Steps

1. Expand automated tests to cover auth, product catalog, checkout, notifications, and i18n snapshots.
2. Complete Stripe production readiness (keys, webhook secrets, signature verification and replay handling checks).
3. Add public producer profile route/page with richer producer portfolio and trust metrics.
4. Add CI checks for locale key parity (English vs Arabic) to prevent translation drift.
5. Prepare deployment manifests for API/web/DB/AI service with environment-specific secrets handling.